ML_Algorithms_Assignments

October 2, 2026

1 Machine Learning Algorithms

1.1 1. K-Means Clustering

Aim: Group data points into clusters using the K-Means algorithm.

[ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

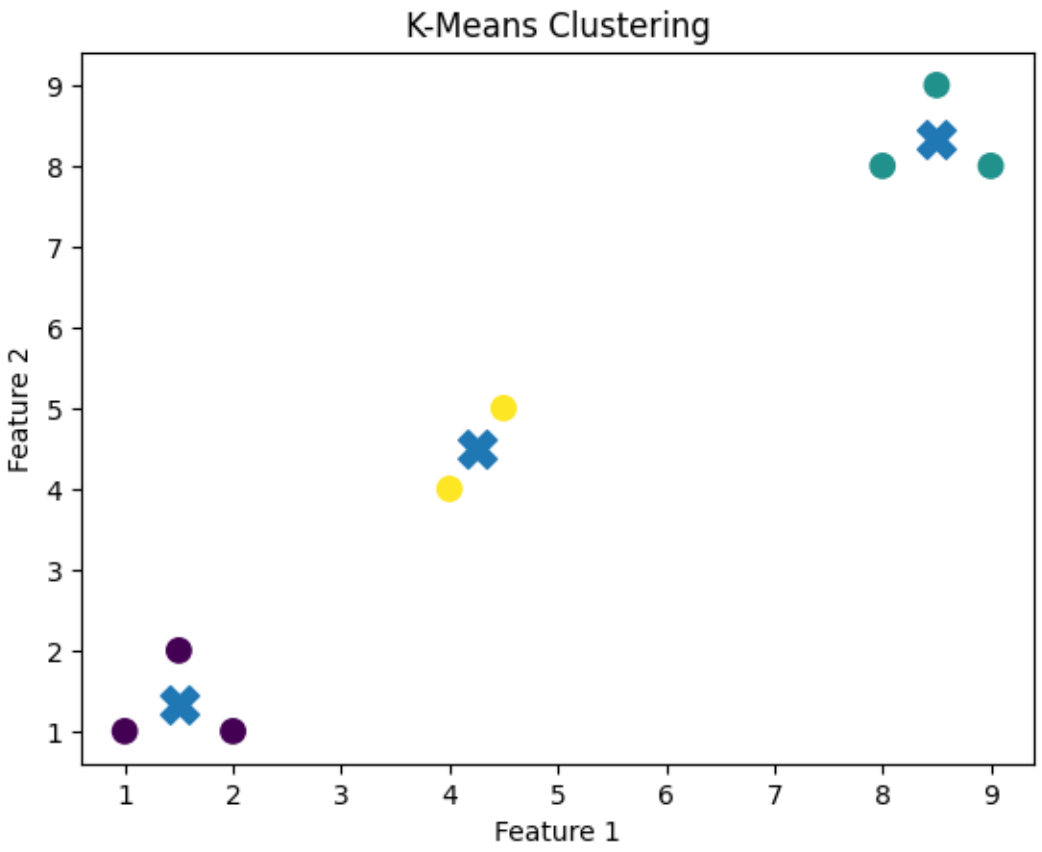
X = np.array([[1,1],[1.5,2],[2,1],[8,8],[8.5,9],[9,8],[4,4],[4.5,5]])

model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)

print("Cluster labels:", labels)
print("Centroids:", model.cluster_centers_)

plt.scatter(X[:,0], X[:,1], c=labels, s=80)
plt.scatter(model.cluster_centers_[:,0], model.cluster_centers_[:,1],␣
 ↪marker='X', s=200)
plt.xlabel('Feature 1'); plt.ylabel('Feature 2'); plt.title('K-Means␣
 ↪Clustering'); plt.show()
```

```
Cluster labels: [0 0 0 1 1 1 2 2]
Centroids: [[1.5        1.33333333]
 [8.5        8.33333333]
 [4.25       4.5       ]]
```



1.2 2. Hierarchical Clustering

Aim: Perform agglomerative hierarchical clustering and visualize the resulting clusters.

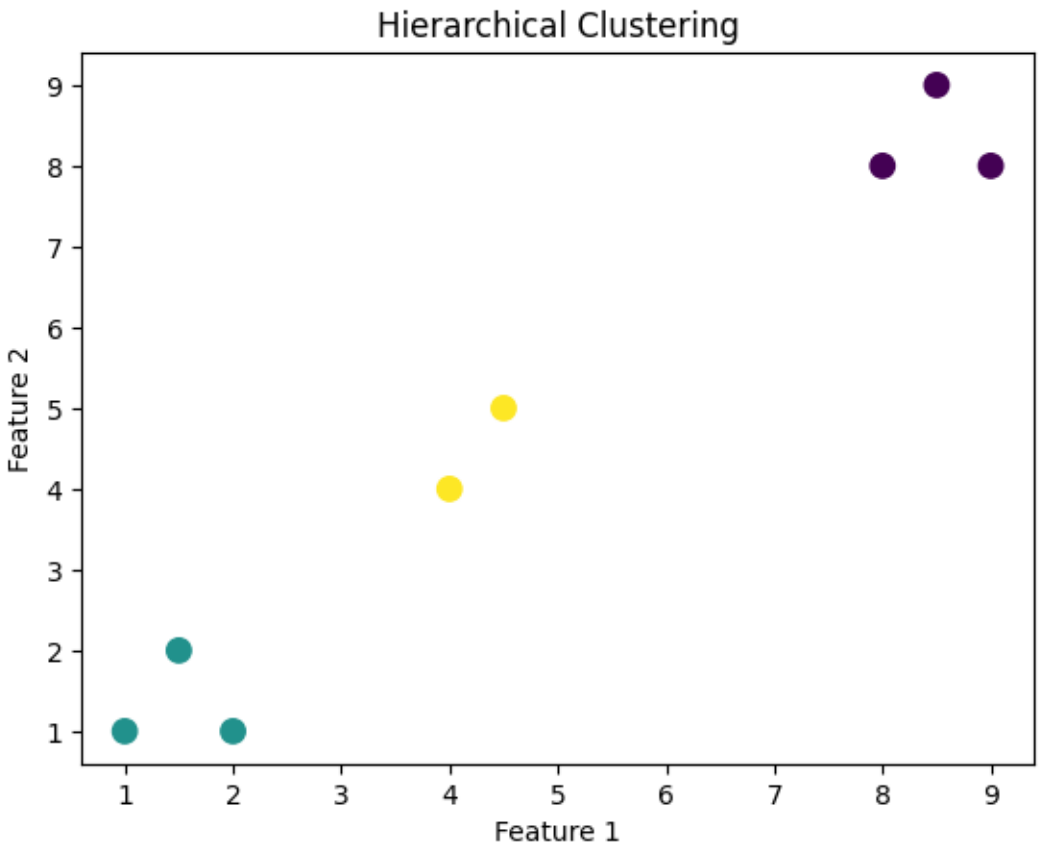
[ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering

X = np.array([[1,1],[1.5,2],[2,1],[8,8],[8.5,9],[9,8],[4,4],[4.5,5]])

model = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = model.fit_predict(X)
print("Cluster labels:", labels)

plt.scatter(X[:,0], X[:,1], c=labels, s=80)
plt.xlabel('Feature 1'); plt.ylabel('Feature 2'); plt.title('Hierarchical␣
↪Clustering'); plt.show()
```

```
Cluster labels: [1 1 1 0 0 0 2 2]
```



1.3 3. Principal Component Analysis (PCA)

Aim: Reduce the dimensionality of a dataset while retaining as much variance as possible.

[ ]:
```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print("Original shape:", iris.data.shape)
print("Reduced shape:", X_pca.shape)
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total variance retained:", pca.explained_variance_ratio_.sum())
```

```
Original shape: (150, 4)
Reduced shape: (150, 2)
Explained variance ratio: [0.72962445 0.22850762]
Total variance retained: 0.9581320720000164
```

1.4 4. Apriori Algorithm

Aim: Find frequent itemsets and association rules from transaction data.

[ ]:
```python
from itertools import combinations
from collections import Counter

transactions = [
    {'Milk','Bread','Butter'},
    {'Bread','Butter'},
    {'Milk','Bread'},
    {'Milk','Bread','Butter','Eggs'},
    {'Bread','Eggs'},
]

min_support = 0.4
N = len(transactions)

def support(itemset):
    return sum(itemset <= t for t in transactions) / N

items = sorted(set().union(*transactions))
frequent = {}
for r in range(1, 4):
    for combo in combinations(items, r):
        s = support(set(combo))
        if s >= min_support:
            frequent[combo] = s

print('Frequent itemsets (support):')
for itemset, s in frequent.items():
    print(itemset, '->', round(s, 2))

print('\nAssociation rules:')
for itemset, s in frequent.items():
    if len(itemset) >= 2:
        S = set(itemset)
        for r in range(1, len(itemset)):
            for antecedent_tuple in combinations(itemset, r):
                antecedent = set(antecedent_tuple)
                consequent = S - antecedent
                conf = s / support(antecedent)
                if conf >= 0.7:
```

```
                    print(set(antecedent), '=>', consequent, 'confidence =',␣
↪round(conf,2))
```

```
Frequent itemsets (support):
('Bread',) -> 1.0
('Butter',) -> 0.6
('Eggs',) -> 0.4
('Milk',) -> 0.6
('Bread', 'Butter') -> 0.6
('Bread', 'Eggs') -> 0.4
('Bread', 'Milk') -> 0.6
('Butter', 'Milk') -> 0.4
('Bread', 'Butter', 'Milk') -> 0.4

Association rules:
{'Butter'} => {'Bread'} confidence = 1.0
{'Eggs'} => {'Bread'} confidence = 1.0
{'Milk'} => {'Bread'} confidence = 1.0
{'Butter', 'Milk'} => {'Bread'} confidence = 1.0
```

1.5 5. Neural Network Basics

Aim: Build a simple neural network classifier and evaluate its accuracy.

[ ]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.
↪target)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print('Test accuracy:', round(accuracy_score(y_test, pred), 4))
print('Predicted classes:', pred)
print('Actual classes:   ', y_test)
```

```
Test accuracy: 0.9667
Predicted classes: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]
Actual classes:    [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
```

[ ]: